# Multicore Processors: Architecture & Programming

# Projects

Mohamed Zahran (aka Z)
Mohamed.zahran@nyu.edu
http://www.mzahran.com

# The project part of the course

- 50% of the total grade
- groups of one or three students
  - One submission per group
- Presentations in the last two lectures of the course
- You have 3 choices:
  1. Pick one of the suggested projects.
  2. Pick one of the suggested projects but <u>suggest</u> a modification.
  3. Suggest your own project.

**For choices 2 and 3, you need to discuss with me first.**

# Milestones

# Milestone 1: Feb 25th (Midnight)

## Between Now and Feb 25th

- You can discuss with me any suggested ideas you may have about the project.

## By midnight of Feb 25th

- Use the link in:

  Brightspace → Content→ Project → Project Selection
- The link directs you to a form.
- This form needs to be filled by each group (i.e. <u>one submission per group</u>).

# Warning!!

- If you do not pick your project by end of Feb 25th → (-2%) of the total project grade for each late day.

- If you decide for a modified version of the published projects without discussing it with me first, the project topic may be rejected because it is too hard/simple, and you start getting late penalty.
  - So, don't wait till Feb 25th to discuss your suggested idea with me.

# Milestone 2: Apr 15th (midnight)

- **Final submission** → 40% of the total course grade
- No specific length for the report.
- Submit through **Brightspace**: one zip-file that contains:
  - report
  - All source code
  - A small text file, called **readme.txt**, telling us how to compile/execute your work (i.e. sort of quick user manual)
  - VERY IMPORTANT: All projects must work on our CIMS machines and does not use any software/library that does not work or is not present on CIMS. Telling us that it works on your laptop does not count.
- **Report Content:**
  - Abstract
  - Introduction
  - Literature Survey
  - Proposed Idea
  - Experimental Setup
  - Experiments & Analysis
  - Conclusions
  - References

# Milestone 3: Apr 22nd and Apr 29th

- **Presentations** (done online) → 10% of the course grade
  - More on the presentation format as we get closer to the presentation date.

# Regarding the Final Report

# Regarding Final Report: Abstract

- The first thing in your report, but the last thing to write.
- 1 paragraph summarizing your problem and why it is important.
- 1-2 lines hinting on your technique
- 1-2 lines summarizing your finding(s)

# Regarding Final Report: Introduction

- Expand the abstract to include why the problem you are solving is important.

- Then, give a general idea about your way of approaching the problem.

# Regarding Final Report: Literature Review

- Feel free to organize this section in any way you want:
  - Include any subsections you think you need.
  - Draw any Figures you like.
- But you need to discuss *at least* the following items:
  - What did others do regarding the topic at hand? Note: the worst survey ever is when you say, "x has done y; z has done k; …". You must put the work of others in taxonomy. That is, "The solution to this falls in x categories: …. . In the first category x has done …".
  - What are the pros and cons of what they are doing?
  - Why their work is not enough, and your proposed project is needed?
  - Important: Literature must not be about papers solving exactly your same problem. But can include papers about different problems but have similar challenges.

# Regarding Final Report: Proposed Idea

- This is a major part of this report.
- Describe in great details the solution to the problem at hand.
  - For example, if you are parallelizing something, you must explain how you did that and justify your choices. If you are comparing things, you must specify what is the criteria of comparison is/are and why you picked them; and so on…

# Regarding Final Report: Experimental Setup

- State:
  - The simulator you used (if any) or the specs of the machine you used
  - The benchmarks (if any),
  - etc …
- How do you know you have written a good experimental setup? If another researcher reads this section, the researcher must be able to replicate your experiments without asking you any questions.

# Regarding Final Report: Results and Analysis

- **Formatting tip**: using tables, bullets, figures, diagrams, … are better than words in presenting results.
- What is/are the measures of success in your work? That is, what do you need to measure and what are you expecting the results to be in order to say that you succeeded?
- What are the experiments that you did in order to get the measurements you mentioned in the above bullet?
- **Analysis:**
  - **Bad**: "As we can see x is increasing with y" ← You won't get any credits for that!
  - **Much better**: "x increases with y because 1, 2, 3, …. And under the conditions …. x may not increase with y. We learn from that …."

# Regarding Final Report: Conclusions

- What are the most important findings of your project?
  - A bullets list is the best here.

- No more than three points.

# Regarding Final Report: References

- If you only mention the references I provided you, it means you did not do any extra work.
- Websites like wikipedia, stackoverflow, etc are not references, don't use them.
- Don't use urls unless there is no paper about the topic.
- Example of reference format:
  - J. Lee and H. Kim. *Tap: A tlp-aware cache management policy for a cpu-gpu heterogeneous architecture*. In High Performance Computer Architecture (HPCA), 2012 IEEE 18th International Symposium on, pages 1–12, 2012.

# Suggested Projects

Check the page on Brightspace → content → projects (links and readings)

# Project:
# Pick an application to parallelize
## (YOU must pick the application.)

- Must demonstrate lessons beyond "I implemented X on the multicore using this library/language/thread model, and it runs faster than a sequential version".
- Better compare with another parallel version in addition to the sequential version.
- Must clearly show the challenges facing the parallelization of this application and how you overcame it.

**These lessons may include:**

- Interesting generalized algorithmic or data structure contributions that can be applied to other applications (and these other applications should be made explicit)
- Interesting optimization techniques that can be applied beyond the application in the paper
- Interesting data management/representation techniques: how to best leverage the memory hierarchy, how to represent data in the most suitable way
- Rigorous comparisons with existing implementations.

# Project:
## Concurrent Data Structure

- Pick a data structure: queue, tree, … , or invent your own.
- Build a library for that data structure that can be used by OpenMP developers.
- Test your implementation for performance and scalability using a set of benchmarks that you create.

# Project:

## Comparing OpenMP and Rust

- **Programmability** (i.e. how easy it is to program with the language)
- **Scalability** (handling large number of threads)
- **Runtime overhead** of the language (measure how long it takes to create threads, terminate threads, synchronize, etc)
- **The amount of control given to the programmer** (can the programmer decide which thread goes to which core, can the programmer decide which variables to be shared and which ones to be priviate, etc).
- Any other factor you can think of.

- You need to create several benchmarks to test the above.
- Based on your analysis about the performance of each language, make a flowchart showing developers when to use each language.

# Project:

## Performance Prediction of Multithreaded Applications (Version A)

- **Input:**
  - Specs of a multithreaded program: what these specs are is part of the project.
  - Specs of a machine X: what are the specs is part of the project.
  - Problem size

- **Output:**
  - Predicted speedup on machine X relative to one thread execution.

# Project:

## Performance Prediction of Multithreaded Applications (Version B)

- **Input:**
  - Specs of a multithreaded program: what these specs are is part of the project.
  - Specs of the machine(s) used to run program: what these specs are is part of the project.
  - Problem size

- **Output:**
  - Predicted speedup relative to a one thread execution on a *different* machine.

# Project:

## Memory Allocator for OpenMP Programs

- Implement a memory allocator for OpenMP multithreaded programs instead of the basic standard one.

- **Output:**
  - Implement your own and compare to at least two others in terms of:
    - Speed
    - Scalability

# Project: Compare Coherence Protocols

- Exhaustive list of coherence protocols
- Implement them using simulator
- Discuss effect on parallel programs in terms of:
  - Scalability
  - Bandwidth requirement

# Project:

## Analyze Real Scalability of Multithreaded Programs

- On Linux
- You create several benchmark programs using OpenMP: compute-bound, memory-bound, and synchronization heavy. Feel free to add more types.
- At least three benchmark programs. But the more the better.
- Calculate speedup from 1 to p cores using Amdahl's law. You need to find a way to calculate F.
- Measure the real speedup on the real machines on 1 to p cores.
- Retrieve hardware counter data (e.g. cache misses, etc) to help you explain the discrepancies between calculated and measured speedup.
- Reach a conclusion that explains when Amdahl's law hold (i.e. within 10% of the measure ones) and when not.

Have Fun!